

Static Ad Generator

System Documentation — how it works & where things live

This document explains the end-to-end static ad generation system: the intake form, the automation pipeline, the brand knowledge base, the template library, the AI image generation, and where every piece of data is stored. It reflects the state of the system as of the July 2026 update cycle.

Prepared: July 15, 2026

Scope: Static Ad Generator (SA-1) — form, n8n pipeline, Airtable, Supabase, KIE AI, Vercel

Audience: Creative Ad·Bundance team (operators & maintainers)

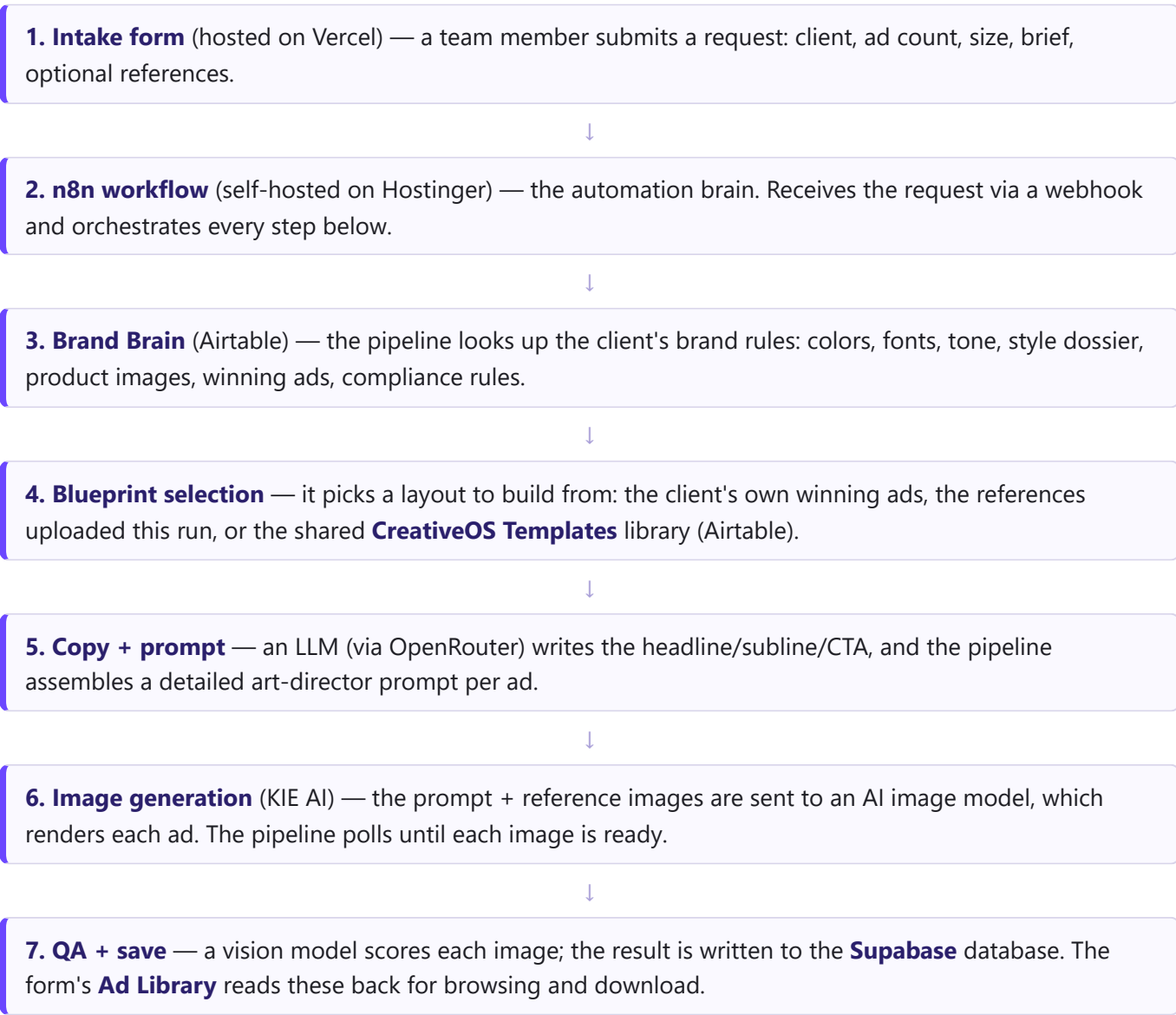
1. Overview

The Static Ad Generator turns a short brief into finished, on-brand static ad creatives for any client. A team member fills in a web form — pick the client, how many ads, the placement size, an optional brief, and optional reference images — and an automated pipeline does the rest: it looks up that client's brand knowledge, picks a proven layout to build from, writes the ad copy, generates the images with AI, quality-checks each one, and saves the results to a library the team can browse and download.

It runs off a single shared **Brand Brain** (about 90 clients), so any client already onboarded can be generated for without re-entering brand details each time.

2. The system at a glance

Seven services work together. The flow of a request:



Where everything lives

Piece	Platform	Location / identifier	What it holds
Static Ad Generator — System Documentation — Creative Ad-Bundance — July 2026			

Intake form	Vercel	<code>form.creativeadbundance.com/forms</code>	The web app that submits requests and browses results
Automation pipeline	n8n (Hostinger)	<code>n8n-i3t9.srv1486031.hstgr.cloud</code>	The "Static Ads Generator" workflow; webhook path <code>/webhook/Static-Ads</code>
Brand Brain	Airtable	<code>base appvCkX59PBphJG0d → table tblIqcPJRvpQhS4AM</code>	Per-client brand knowledge (86 rows)
Template library	Airtable	<code>same base → table tblE6i41y5UUClbVW</code>	972 proven ad layouts ("CreativeOS Templates")
Generated ad records	Supabase	<code>project xakngjsybyytlidyqfsmi → table static_ads</code>	One row per generated ad (URL, QA score, metadata)
Uploaded assets	Supabase Storage	<code>buckets: product-images, ad-references</code>	Product photos and per-run reference uploads
Image generation	KIE AI	<code>api.kie.ai</code>	Renders the images (GPT Image 2 / Nano Banana Pro)
Copy & QA models	OpenRouter	<code>openrouter.ai</code>	Claude/GPT for copywriting and visual QA scoring

3. The components in detail

3.1 The intake form

A single-page web app. The source is one self-contained HTML file (`Docs/Static Ads Generator/static_ads.html.md` in the repo) and it is **hosted on Vercel** (project `static-ads-intake`) at `form.creativeadbundance.com/forms` . It was previously served from GoHighLevel; the domain's DNS (managed at Squarespace) now points to Vercel.

What the form captures

- **Client name** — typed, or picked from a "Browse Clients" list of ~94 clients pulled from the Brand Brain.
- **Number of static ads** — how many layouts to build (see the variants note below).
- **Generation mode** — Top performers (shared templates), Client's winning ads (upload), My reference (upload), or Team's top 5 (winning ads already on file).
- **Platform / size** — 1:1 (Meta/TikTok square), 9:16 (vertical), 1.91:1 or 16:9.
- **Brief** — free text direction.
- **Reference images / avatar** — optional uploads. These go straight into Supabase Storage (the `ad-references` bucket) and only the URLs are sent onward.

Login is handled by Supabase auth. On submit, the form POSTs the request to the n8n webhook. The form also contains an **Ad Library** view that reads generated ads back from Supabase and groups them by brand and batch.

Client list: the picker is a fixed list baked into the form, generated from the Brand Brain (union of every `client_name` and `brand_name`). New clients added to the Brain later do not appear automatically — the list must be regenerated (a 30-second task) or, as a future improvement, changed to fetch live.

3.2 The Brand Brain (Airtable)

This is the heart of the system: one Airtable row per client holding everything the AI needs to stay on-brand. The pipeline's **Search Brand Brain** step looks up the submitted client and feeds these fields into the prompt.

Key fields

- **client_name / brand_name** — identity. Some clients are holding companies with several product brands (e.g. Scale Media → LiverMD, MoveMD...); the lookup matches either field.
- **brand_guidelines** — a detailed "style dossier": layout patterns, typography, exact color usage, product treatment, recurring devices, and a "never-do" list. This is the single biggest driver of on-brand output.
- **Colors** (primary/secondary/accent hex) and **brand_fonts**.
- **brand_tone, brand_personality, target_personas, key_offer, product_benefits, core_pain_points** — the messaging inputs.
- **compliance_notes / compliance_disclaimer** — mandatory legal rules (critical for regulated clients such as debt, health, and supplement brands).
- **logo_urls, product_image, winning_ads** — image attachments. **product_image** is the packshot the AI must reproduce; **winning_ads** are the client's own proven creatives used as blueprints.

Two fields do the heavy lifting for quality: a rich **brand_guidelines** dossier makes the design on-brand, and a **clean, isolated product_image** (not a busy promo composite) makes the product render faithfully. Brands missing either will produce weaker results.

3.3 The template library (CreativeOS Templates)

A second Airtable table holding **972 proven ad layouts** (54 original + 918 imported from the "Mr. Paid Social" swipe library). Each row is a template image tagged with an aspect ratio and category. When a run has no client references or winning ads to clone, the pipeline pulls layouts from here as the blueprint. The pipeline only queries the aspect ratio requested (**1:1** , **9:16** , or **16:9**).

3.4 The n8n pipeline

The workflow ("Static Ads Generator") runs these nodes in order:

Node	What it does
Webhook1	Receives the form submission (client, count, size, brief, reference URLs).
Decode Reference	Reads any reference image URLs uploaded this run.
Search Brand Brain	Looks up the client's row in Airtable and loads all brand fields.
Search records	Pulls candidate templates from CreativeOS Templates, filtered to the requested aspect ratio.
Shuffle / Pick Templates	Chooses the blueprint per ad. Priority: client-uploaded references → the brand's own winning ads → shared templates.
Build KIE AI Prompt	Assembles the full art-director prompt per ad: the blueprint's role, a product-fidelity lock, brand colors/fonts/guidelines, compliance rules, and layout guardrails. Produces one prompt per layout × copy angle.
Loop Over Items	Processes each ad one at a time.
Generate Ad Copy	An LLM (OpenRouter, Claude Sonnet) writes the headline / subline / CTA and appends it to the prompt.
Create KIE AI Task	Sends the prompt + reference images to KIE AI to render the image.
Wait → Poll → If	Waits and polls KIE until the image reports "success," then continues.
Extract Image URL	Pulls the finished image URL out of the KIE response.
QA Pass → Merge QA	A vision model (Claude Haiku) scores the image 1–10 and flags issues (garbled text, wrong packaging, off-brand color, etc.).
Post to supabase	Inserts the ad record (URL, indices, platform, QA score/flags) into the <code>static_ads</code> table.

Credentials (Airtable token, KIE key, OpenRouter key, Supabase service key) are stored inside the n8n nodes / credential store. The workflow itself lives in n8n cloud, not in git; the repo keeps exported copies and node source under `Docs/Static Ads Generator/`.

3.5 Image generation (KIE AI)

Images are rendered through **KIE AI** (`api.kie.ai`), which resells several top image models. Two are relevant:

- **GPT Image 2** — very faithful to the reference product, but tends to be literal and low-variety.
- **Nano Banana Pro** — much richer concept variety and premium photography; keeps the product faithful *when* given a clean product image plus a "reproduce exactly" instruction.

Important gotcha: KIE result URLs expire after roughly 24 hours. Unless the finished image is downloaded and re-hosted, saved records will show broken images the next day. Making the pipeline download & re-host outputs (to Supabase Storage) before saving is the fix for durable libraries.

3.6 Output storage & the Ad Library (Supabase)

Every generated ad is a row in the Supabase `static_ads` table (project `xakngjsybyytldyqfsmi`). Each row stores the brand name, the image URL, the variant and template indices, the platform/aspect, a run ID, and the QA score, flags, and notes. The form's **Ad Library** queries this table to show ads grouped by brand and batch, with filters (ship-ready 8+ vs needs-review) and a download option.

4. How a single run works, step by step

1. A team member opens the form, picks a client (e.g. *Smooth Sailing*), sets the count and size (e.g. 3 × 1:1), adds a brief, and submits.
2. n8n receives it and loads Smooth Sailing's Brand Brain row — its orange/cream palette, retro-beach dossier, product pouch image, tone, and compliance rules.
3. It picks blueprints: if references were uploaded, those; else the brand's winning ads; else shared templates that match 1:1.
4. For each blueprint (and each of the copy angles), it writes copy and builds a full prompt that locks the exact product, applies the brand skin, and translates any foreign template props into the brand's world.
5. It sends each prompt + the product image to KIE AI, waits for the render, and pulls the image URL.
6. A vision model scores each image; the row is written to Supabase.
7. The team opens the Ad Library, filters to Smooth Sailing's newest batch, reviews, and downloads the winners.

5. What changed in the July 2026 update cycle

The recent work fixed the two biggest complaints (wrong product, off-brand/samey design) and moved the form to a maintainable host. Status is marked below.

Change	Why	Status
Full brand dossier reaches the render	Only a fragment of <code>brand_guidelines</code> was being used, so output ignored the brand's colors, fonts, and formats.	Delivered (paste into n8n)
Product-form lock	The AI turned pouches into jars. Now the product's form comes only from the real photo.	Delivered
Auto-clone the brand's own winning ads	Cloning a proven layout is what reaches designer quality. Now automatic when no references are uploaded.	Delivered
Concept translation	Templates were copied literally (e.g. a cookware "before/after" kept the cookware). Now the concept is re-mapped to the brand.	Delivered
Copy fix + 2 variants per layout	The copy step's API key was a placeholder (silent failure); variant count made explicit.	Delivered
Match client OR brand name	Clients whose value lives in the <code>brand_name</code> column were unfindable.	Delivered
Nano Banana Pro + clean packshots	Proven to keep the product faithful while adding concept variety. Recommended render model + data task.	Recommended
Form moved to Vercel	Off GoHighLevel; enables versioned, one-command deploys.	Live
Form scroll/routing bugs fixed	The request form could not scroll and the gallery painted over it.	Live

Client picker expanded to 94	Was 48; now every Brand Brain client is selectable.	Live
------------------------------	---	------

6. Known limitations & next steps

- **Output images expire (~24h).** The pipeline saves KIE's temporary URL directly, so the Ad Library shows broken images after a day. Fix: download & re-host each output to Supabase Storage before saving.
- **Failed renders can stall a run.** The polling step only exits on "success," so a failed KIE job re-polls until the run times out, producing fewer ads than requested. Fix: add failure/timeout handling with retry-or-skip.
- **"Generation mode" is cosmetic.** The workflow does not read the mode dropdown; behavior is driven by whether references are uploaded (plus winning-ads auto-sourcing). Wire it up if the modes should differ.
- **Clean packshots are a data task.** Product fidelity depends on each brand having a clean, isolated product photo in the Brain — not a promo composite.
- **winning_ads coverage is thin.** Only ~6 brands have their proven ads loaded; the rest fall back to the shared library. Loading each brand's best 4–6 statics unlocks the clone-a-winner quality.
- **Form deploys are manual for now.** Connecting the Vercel project to the repo (with the form folder as its root) would make future fixes a `git push`.

7. Quick reference

Item	Value
Live form	https://form.creativeadbundance.com/forms
Vercel project	static-ads-intake (also <code>static-ads-intake.vercel.app</code>)
Form source (repo)	<code>Docs/Static Ads Generator/static ads.html.md</code>
n8n host / webhook	<code>n8n-i3t9.srv1486031.hstgr.cloud</code> • <code>POST /webhook/Static-Ads</code>
Airtable base	<code>appvCkX59PBphJG0d</code> ("Creative Adbundance - Batch Records")
— Brand Brain table	<code>tblIqcPJRpQhS4AM</code> (86 rows)
— CreativeOS Templates table	<code>tblE6i41y5UUCbVW</code> (972 rows)
Supabase project	<code>xakngjsybyytldyqfsmi</code>
— Output table	<code>static_ads</code>
— Storage buckets	<code>product-images</code> , <code>ad-references</code>
Image generation	KIE AI (<code>api.kie.ai</code>) – GPT Image 2 / Nano Banana Pro
Copy & QA models	OpenRouter (Claude Sonnet for copy, Claude Haiku for QA)
Repo docs folder	<code>Docs/Static Ads Generator/</code> (workflow exports, node source, this doc)

Note on credentials: all API keys and tokens (Airtable, KIE, OpenRouter, Supabase service role) are stored inside the n8n credential store and node configurations. They are intentionally not reproduced in this document.